

## 0.1 S7comm detection rule design approach

This chapter presents the Suricata rule set for detecting attack behavior on the Siemens S7comm protocol, based on the traffic analysis in Chapter ???. The approach belongs to the signature-based detection group: each rule matches a fixed combination of bytes in the TCP/102 payload (TPKT, COTP, S7 Header, function code).

All rules live in `detect/rules/s7comm.rules`, `detect/rules/ics_dos.rules`, and `detect/rules/s7comm_malformed.rules`. The `sid` numbers are grouped as follows: 100000x (session setup, reconnaissance), 100001x (read/write variables), 100002x (download/upload blocks), 100003x (Start/Stop control), 100004x (DoS / rate flood), 100005x (malformed / abnormal syntax).

Designing a rule is not a direct copy of a Wireshark dissector line. Each signature must state which layer is checked (TPKT, COTP, S7 header), which direction the packet travels (client→PLC or PLC→client), and whether a single byte match is enough or a rate threshold is needed. The sections below follow that order: first the shared byte layout, then the workflow from PCAP to rule, then one subsection per attack scenario from Chapter 3.

## 0.2 From captured traffic to Suricata signatures

Every rule in this chapter was derived from PCAP files recorded while running the Chapter 3 scenarios. The same five-step process was applied to each scenario group (reconnaissance, Start/Stop replay, upload/download, Write Var / MITM, DoS, malformed).

1. Run the scenario from Chapter 3 and record traffic. Each subsection in Chapter ??? has a script or tool (`nmap --script s7-info`, `scanBlock.py`, TIA Portal replay, `s7comm_dos.py`, `s7comm_malformed.py`, Snap7 Write Var, Bettercap MITM). Capture on the IDS mirror interface or on the attacker host so the PCAP contains the full TCP/102 payload.
2. Open the PCAP in Wireshark and locate TPKT, COTP, and S7comm in the hex dump. Valid S7comm Data TPDU's start with 03 00 (TPKT version 3) at offset 0, 02 F0 80 (COTP Data TPDU) at offset 4, and 32 (S7 protocol id) at offset 7. If these bytes are missing, the packet is either not S7comm or belongs to the malformed group (Section ???).
3. Choose flow direction and locate the function byte. Client-initiated Jobs use `ROSCTR=0x01` (|02 F0 80 32 01| at offset 4); the function code sits at offset 17. Userdata requests (Read SZL, List blocks) use `ROSCTR=0x07`; parameters still start at offset 17. PLC responses with `ROSCTR=0x03` (Ack\_Data) place the function code at offset 19 because two error-class bytes precede the parameter block. Suricata `flow:to_server` or `flow:from_server` must match this direction before the content match is evaluated.
4. Decide between a single `content` match and a `threshold`. One Setup Communication or one Write Var in a session is normal engineering traffic; the rule fires on the first match (`sid:1000001`, `sid:1000010`). DoS scripts send the same valid PDU dozens of times per second; those signatures reuse the same byte pattern but add `threshold:type both, track`

`by_src`, `count N`, `seconds 10` so a single HMI poll does not alert. Malformed samples change one field while keeping the rest valid; a `content` or `byte_test` on that field is enough without a threshold.

5. Validate offline with `suricata -r scenario.pcap -S detect/rules/s7comm.rules` (and the DoS or malformed rule files when needed). Compare `fast.log` alert `sid` values and timestamps with Wireshark packet numbers. If an expected packet does not alert, check offset (17 vs 19), flow direction, or whether the test PCAP was captured on the correct interface.

This workflow explains why two rules can share the same function byte (0x1b appears in both `sid:1000021` and `sid:1000026`) yet differ in `flow` and `ROSCTR`: they describe opposite directions in the download handshake observed in Section ??.

## 0.3 Common rules for identifying the S7comm protocol

S7comm on TCP/102 is structured as `TCP → TPKT → COTP → S7comm`. Table ?? lists the byte offsets used in the rules.

The TPKT length field (bytes 2–3, big-endian) is not used in the opcode rules but appears in malformed detection (`sid:1000051`, `sid:1000058`) where the declared length disagrees with the actual TCP segment size. Keeping offsets fixed across all rule files avoids re-counting bytes when moving from Wireshark to Suricata syntax.

### 0.3.1 Flow direction and PDU type conditions

In the lab environment, the PLC listens on TCP/102 and is modeled as the server; the HMI and the attacker act as clients. Suricata applies two condition layers before matching a specific function code: (i) a `flow` condition that fixes direction relative to the TCP/102 server; (ii) a `content` condition that fixes the S7comm PDU type (Job, Userdata, or Ack\_Data) and the function byte position.

**Job PDU — client → PLC direction.** Client-initiated operations (Setup Communication, Write Var, Request Download, PLC Control, ...) are sent as Job PDUs with `ROSCTR=0x01`. The matching rules use:

- `flow:to_server,established`: limit inspection to packets the client sends to port 102 on `$HOME_NET` within an established TCP session;
- `content:"|02 F0 80 32 01|"; offset:4; depth:5`: confirm the COTP Data TPDU header (02 F0 80), S7 protocol id (32), and Job `ROSCTR` (0x01).

For Userdata commands (Read SZL, List blocks), `ROSCTR` is 0x07; the pattern at offset 4 becomes `|02 F0 80 32 07|`. For both Job and Userdata, the function code byte is at offset 17 (Table ??).

**Ack\_Data PDU — PLC → client direction.** PLC responses (write confirmations, block data returns, ...) use R0SCTR=0x03. The matching rules use:

- **flow:from\_server,established:** limit inspection to packets originating from the PLC on port 102;
- **content:"|02 F0 80 32 03|"; offset:4; depth:5:** confirm an Ack\_Data PDU.

Unlike Job/Userdata, the Parameter section of Ack\_Data follows two bytes for error class and error code, so the function code sits at offset 19 instead of offset 17. This convention is used consistently in rules that observe the response direction (for example **sid:1000026**, **sid:1000013**).

These three principles filter out non-S7comm traffic on TCP/102, separate request from response direction, and place **content** checks at the correct offset before defining scenario-specific signatures in Section ??.

## 0.4 Rules by attack scenario

### 0.4.1 Reconnaissance

Based on Section ??: TCP/102 scanning (Nmap **s7-info**), Read SZL (0x0011, 0x001c), and block listing (Snap7).

**Link to Chapter 3 traffic.** After the TCP and COTP handshake, Nmap and Snap7 both send Setup Communication with function 0xF0 at offset 17 on a Job PDU (0300...02f0803201...f0 visible in the reconnaissance PCAP). Read SZL requests then appear as Userdata (R0SCTR=0x07) with parameter head 00 01 12 04 11 44 01 00 at offset 17—Wireshark labels this as CPU functions / Read SZL. Nmap **s7-info** reads SZL-ID 0x0011 (module identification, bytes 00 11 00 01 in the data section) and 0x001c (communication status, 00 1C 00 00), which map directly to the Nmap text fields Module, Serial Number, and System Name. Block inventory from **scanBlock.py** uses subfunction List blocks with parameter block 00 01 12 04 11 13 01 00 instead of ...44.... These fixed byte strings are stable across OpenPLC and real S7 stacks, so they become **content** anchors rather than IP-based rules.

**Rule sid:1000001.**

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM Setup Communication to PLC";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|F0|";
  offset:17; depth:1; sid:1000001; rev:1;
  metadata:service iso-tsap, protocol s7comm, attack reconnaissance; priority:3;)
```

**From traffic to rule.** In the reconnaissance PCAP, the first S7comm Job after COTP setup carries 0xF0 at offset 17 while bytes 4–8 read 02 F0 80 32 01. That byte is the Setup Communication function code, required before any SZL or block query. The rule chains TPKT, Job header, and |F0| with **flow:to\_server** because only the scanner client sends this PDU to the PLC. It fires on every new session setup, including legitimate HMI connections, so it is a context alert (priority 3) rather than a standalone attack indicator.

#### Rule sid:1000002.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM Read SZL request - PLC
information enumeration"; flow:to_server,established; content:"|03 00|";
offset:0; depth:2; content:"|02 F0 80 32 07|"; offset:4; depth:5;
content:"|00 01 12 04 11 44 01 00|"; offset:17; depth:8; sid:1000002; rev:1;
metadata:service iso-tsap, protocol s7comm, attack reconnaissance; priority:2;)
```

**From traffic to rule.** Wireshark decodes Read SZL as Userdata with ROSCTR=0x07 and an eight-byte parameter prefix 00 01 12 04 11 44 01 00 (method 0x11, function group CPU functions, subfunction Read SZL). Nmap s7-info emits this pattern immediately after Setup Communication in Section ???. Matching the full eight-byte block at offset 17 avoids confusing Read SZL with other Userdata services that share 32 07 but use a different subfunction byte. The rule does not distinguish which SZL-ID is requested; that refinement is left to sid:1000003 and sid:1000004.

#### Rule sid:1000003.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM Read SZL module identification
0x0011"; flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
content:"|02 F0 80 32 07|"; offset:4; depth:5;
content:"|00 01 12 04 11 44 01 00|"; offset:17; depth:8;
content:"|00 11 00 01|"; distance:0; within:12; sid:1000003; rev:1;
metadata:service iso-tsap, protocol s7comm, attack reconnaissance; priority:2;)
```

**From traffic to rule.** In packet 1048 of the Nmap capture (Section ??), the data section after the Read SZL parameter contains SZL-ID 0x0011 and index 0x0001, which Wireshark shows as |00 11 00 01| within twelve bytes of the parameter head. That read returns the module order number that appears in Nmap output as Module: 6ES7 315-2EH14-0AB0. The rule adds this four-byte constraint with distance:0; within:12 so generic Read SZL probes (sid:1000002) and module-fingerprint reads are separated. Engineering tools may also read 0x0011 during commissioning, so the alert should be correlated with source address and timing.

#### Rule sid:1000004.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM Read SZL communication status
0x001c"; flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
content:"|02 F0 80 32 07|"; offset:4; depth:5;
content:"|00 01 12 04 11 44 01 00|"; offset:17; depth:8;
content:"|00 1C 00 00|"; distance:0; within:12; sid:1000004; rev:1;
metadata:service iso-tsap, protocol s7comm, attack reconnaissance; priority:2;)
```

**From traffic to rule.** The next Nmap probe in the same session requests SZL-ID 0x001c (communication status partial list), encoded as |00 1C 00 00| after the Read SZL parameter block. The PLC reply supplies serial number, system name, and module type strings that Nmap prints in its s7-info table. This rule targets that specific enumeration step rather than every SZL read. It will not fire on 0x0011 reads matched by sid:1000003, but any tool that reads 0x001c for diagnostics will also match.

**Rule sid:1000005.**

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM List Blocks request - PLC block
inventory enumeration"; flow:to_server,established; content:"|03 00|";
offset:0; depth:2; content:"|02 F0 80 32 07|"; offset:4; depth:5;
content:"|00 01 12 04 11 13 01 00|"; offset:17; depth:8; sid:1000005; rev:1;
metadata:service iso-tsap, protocol s7comm, attack reconnaissance; priority:2;)
```

**From traffic to rule.** Snap7 .list.blocks() in scanBlock.py sends User-data with function group Block functions and subfunction List blocks; the eight-byte parameter at offset 17 is |00 01 12 04 11 13 01 00|, differing from Read SZL only in the subfunction byte (0x13 vs 0x44). Wireshark labels this as a block inventory request. An attacker mapping OB/DB/FC/FB before upload or download will trigger this rule after Setup Communication. TIA Portal block browsing uses the same S7comm service, so the alert alone does not prove malicious intent.

## 0.4.2 Start/Stop PLC command replay

Based on Section ??: replay of PLC Control 0x28 and PLC Stop 0x29 payloads with PI-Service P.PROGRAM.

**Link to Chapter 3 traffic.** TIA Portal Start CPU and Stop CPU packets were exported as fixed hex strings in Section ?. Both begin with 0300...02f0803201 (TPKT + COTP + Job header). At offset 17, Start uses function 0x28 and Stop uses 0x29. The ASCII string P.PROGRAM (50 5F 50 52 4F 47 52 41 4D in hex) appears in the PI-Service data within 24 bytes after the function byte—visible in Wireshark as the program name field. Replay on OpenPLC sends the same bytes without TIA authentication. Rules sid:1000030/sid:1000032 match only the function byte; sid:1000031/sid:1000033 add the P.PROGRAM string to match the lab replay payloads exactly.

**Rule sid:1000030.**

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM PLC Control command";
flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|28|";
offset:17; depth:1; sid:1000030; rev:1;
metadata:service iso-tsap, protocol s7comm, attack plc_control; priority:1;)
```

**From traffic to rule.** The captured Start CPU payload places 0x28 (PLC Control) at offset 17 on a Job PDU directed to port 102. The same function code is used for several PI-Services besides cold start—activate block, copy RAM to ROM, and others—so the rule intentionally omits parameter bytes. It gives early warning whenever any Control command reaches the PLC from an unexpected client. Legitimate engineering stations also send 0x28 during commissioning, which limits specificity compared to sid:1000031.

**Rule sid:1000031.**

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM replayed Start CPU P.PROGRAM";
flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|28|";
```

```
offset:17; depth:1; content:"P_PROGRAM"; distance:0; within:24;
sid:1000031; rev:1;
metadata:service iso-tsap, protocol s7comm, attack start_stop_replay; priority:1;)
```

**From traffic to rule.** The lab Start replay hex string in Section ?? contains 0x28 followed within 24 bytes by the ASCII PI-Service name P\_PROGRAM, which selects the CPU program for start. Wireshark shows this as PLC Control with program name P\_PROGRAM. The rule inherits the 0x28 match from sid:1000030 and adds a string match for the replay payload copied from TIA Portal. Other Control commands that use a different PI-Service name will not match, which reduces false positives from benign 0x28 traffic.

**Rule sid:1000032.**

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM PLC Stop command";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|29|";
  offset:17; depth:1; sid:1000032; rev:1;
  metadata:service iso-tsap, protocol s7comm, attack plc_stop; priority:1;)
```

**From traffic to rule.** Stop CPU replay uses function 0x29 (PLC Stop) at offset 17 on the same Job header pattern 02 F0 80 32 01. In the Stop hex dump (0300002102f080...29...), byte 17 is 0x29 regardless of PI-Service details. This rule alerts on any Stop request from client to PLC before checking whether the payload matches the recorded TIA Portal replay. Emergency stop from an authorized engineering station will also trigger it.

**Rule sid:1000033.**

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM replayed Stop CPU P_PROGRAM";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|29|";
  offset:17; depth:1; content:"P_PROGRAM"; distance:0; within:24;
  sid:1000033; rev:1;
  metadata:service iso-tsap, protocol s7comm, attack start_stop_replay; priority:1;)
```

**From traffic to rule.** The Stop replay payload in Section ?? pairs 0x29 with P\_PROGRAM in the same relative position as the Start packet—Wireshark shows param.length = 6 and the program name field. Matching both bytes narrows the alert to the lab replay script rather than any Stop command. If an attacker replays a Stop PDU without the P\_PROGRAM string, only sid:1000032 fires.

### 0.4.3 Block upload and download

Based on Section ??: function sequence 0x1a–0x1f on Siemens traffic or Wireshark simulation.

**Link to Chapter 3 traffic.** Upload and download sessions in Section ?? follow the Siemens function sequence visible in Wireshark: Request Download 0x1a, Download Block 0x1b, Download Ended 0x1c, Start Upload 0x1d, Upload Block 0x1e, End Upload 0x1f. Each appears at offset 17 on Job PDUs from client to PLC, except when the PLC sends block data back—then the client responds with Ack\_Data (ROSCTR=0x03) and 0x1b moves to offset 19

(sid:1000026). The simulated Wireshark capture also shows the PLC initiating Download Block (0x1b, Job, from\_server) in sid:1000021. Because OpenPLC Editor loads programs via REST rather than S7comm, these rules were tuned on TIA Portal / PLCSIM traffic and the documented hex sequences rather than on OpenPLC upload PCAPs.

#### Rule sid:1000020.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM Request Download block to PLC";  
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;  
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|1A|";  
  offset:17; depth:1; sid:1000020; rev:1;  
  metadata:service iso-tsap, protocol s7comm, attack program_download; priority:1;)
```

**From traffic to rule.** The first Job in a download sequence carries function 0x1a (Request Download) at offset 17 after the standard 02 F0 80 32 01 header. Wireshark labels this as the client asking the PLC to prepare for incoming block data. The rule uses `flow:to_server` because only the engineering station sends Request Download. Normal program loads from TIA Portal begin with the same byte, so source IP and session context matter for interpretation.

#### Rule sid:1000021.

```
alert tcp $HOME_NET 102 -> any any (msg:"ICS S7COMM PLC requests Download Block data";  
  flow:from_server,established; content:"|03 00|"; offset:0; depth:2;  
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|1B|";  
  offset:17; depth:1; sid:1000021; rev:1;  
  metadata:service iso-tsap, protocol s7comm, attack program_download; priority:1;)
```

**From traffic to rule.** During download, the PLC sometimes sends a Job PDU with 0x1b (Download Block request) from port 102 toward the client—the opposite direction of sid:1000026. Wireshark shows this as the PLC asking for the next chunk of program data. The rule reverses source and destination (`$HOME_NET 102 -> any any`) and uses `flow:from_server` with `ROSCTR=0x01` and offset 17. Without the direction constraint, the same 0x1b byte would match both handshake legs.

#### Rule sid:1000026.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM client sends Download Block data  
to PLC"; flow:to_server,established; content:"|03 00|"; offset:0; depth:2;  
  content:"|02 F0 80 32 03|"; offset:4; depth:5; content:"|1B|";  
  offset:19; depth:1; sid:1000026; rev:1;  
  metadata:service iso-tsap, protocol s7comm, attack program_download; priority:1;)
```

**From traffic to rule.** When the client pushes block bytes to the PLC, the PDU type switches to Ack\_Data (32 03) and the function 0x1b shifts to offset 19 because of the two-byte error header. This packet carries the actual program content in the data section. The pair sid:1000021/sid:1000026 mirrors the two directions observed in the download sequence diagram of Section ???. The rule does not inspect block content—only the presence of a Download Block transfer.

#### Rule sid:1000022.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM Download Ended";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|1C|";
  offset:17; depth:1; sid:1000022; rev:1;
  metadata:service iso-tsap, protocol s7comm, attack program_download; priority:1;)
```

**From traffic to rule.** Download Ended (0x1c) closes the download session on a Job PDU from client to PLC. In Wireshark it appears after one or more Download Block exchanges. Matching |1C| at offset 17 marks completion of a program push. A single sid:1000020 without follow-up sid:1000026/sid:1000022 may be an aborted download rather than a full compromise.

#### Rule sid:1000023.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM Start Upload block from PLC";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|1D|";
  offset:17; depth:1; sid:1000023; rev:1;
  metadata:service iso-tsap, protocol s7comm, attack program_upload; priority:1;)
```

**From traffic to rule.** Start Upload (0x1d) opens an upload session: the client requests a block name such as \_0800001P from the PLC. The function byte at offset 17 is 0x1d on a Job PDU. This is the first step when an attacker extracts logic from the device. Authorized backup operations from TIA Portal send the same PDU, so the alert indicates exfiltration only when the source is not the known engineering station.

#### Rule sid:1000024.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM Upload Block request";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|1E|";
  offset:17; depth:1; sid:1000024; rev:1;
  metadata:service iso-tsap, protocol s7comm, attack program_upload; priority:1;)
```

**From traffic to rule.** Upload Block (0x1e) repeats while the client pulls data chunks from the PLC; each request is a Job with 0x1e at offset 17. Wireshark shows multiple 0x1e/Ack\_Data pairs until the block is complete. The rule counts each request separately, so a full upload generates many alerts. It does not read the ASCII block data returned by the PLC.

#### Rule sid:1000025.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM End Upload";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|1F|";
  offset:17; depth:1; sid:1000025; rev:1;
  metadata:service iso-tsap, protocol s7comm, attack program_upload; priority:1;)
```

**From traffic to rule.** End Upload (0x1f) terminates the upload session on a Job PDU. It typically follows a burst of sid:1000024 alerts in the same TCP session. Together, sid:1000023–sid:1000025 outline the upload timeline visible in Section ???. The rule alone does not reveal which block was copied.



#### 0.4.4 Process variable writes and MITM observation

Based on Section ??: process writes via Write Var (for example DB1.DBW2, iSetpoint 0x04b0→0x07d0); Ack\_Data interception toward the HMI. Rule sid:1000011 matches any write to the DB memory area without fixing a DB number.

**Link to Chapter 3 traffic.** The Stuxnet lab script sends Write Var (0x05 at offset 17) to change iSetpoint at DB1.DBW2 from 1200 (0x04b0) to 2000 (0x07d0). In the Snap7 PCAP, the Job PDU contains 05 followed by request items: Transport size 0x12, length 0x10, and area byte 0x84 (DB area) at a fixed offset after the function code—the pattern |12|...|10|...|84| used in sid:1000011. The MITM leg does not alter request opcodes; Bettercap and s7\_intercept.py rewrite Read Var responses on the PLC→HMI path, increasing the rate of Ack\_Data (R0SCTR=0x03) from port 102 without a single suspicious function byte. That traffic pattern motivates sid:1000013 with a threshold rather than a content match on modified payload bytes.

**Rule sid:1000010.**

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM Write Var request to PLC";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|05|";
  offset:17; depth:1; sid:1000010; rev:1;
  metadata:service iso-tsap, protocol s7comm, attack modify_process_variable;
  priority:1;)
```

**From traffic to rule.** Every Snap7 or HMI write in the lab places function 0x05 (Write Var) at offset 17 on a Job PDU to port 102. Wireshark expands the following bytes as variable items (area, DB number, address, data). The rule matches only the function byte, so it covers writes to M, I, Q, and DB alike. It is the broad base layer beneath the DB-specific sid:1000011 and will fire on normal HMI setpoint updates unless filtered by source IP.

**Rule sid:1000011.**

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM Write Var to DB memory area";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|05|";
  offset:17; depth:1; content:"|12|"; offset:19; depth:1;
  content:"|10|"; offset:21; depth:1; content:"|84|";
  distance:5; within:1; sid:1000011; rev:2;
  metadata:service iso-tsap, protocol s7comm, attack modify_process_variable;
  priority:1;)
```

**From traffic to rule.** In the Write Var PDU for DB1.DBW2, Wireshark shows an Any-type request item with Transport size 0x12, length 0x10, and area code 0x84 (S7 DB area) five bytes after the Syntax ID field. The rule checks |12|, |10|, and |84| at offsets 19–21 plus distance:5; within:1 for the area byte without hard-coding DB number or offset. The lab Stuxnet write to DB1.DBW2 matches; writes to M/I/Q using area 0x83/0x81/0x82 do not. It still cannot tell legitimate DB tuning from the attack write without context.

**Rule sid:1000013.**

```
alert tcp any 102 -> $HOME_NET any (msg:"ICS S7COMM suspicious Ack_Data response
to HMI/engineering station"; flow:from_server,established;
content:"|03 00|"; offset:0; depth:2; content:"|02 F0 80 32 03|";
offset:4; depth:5; sid:1000013; rev:1;
metadata:service iso-tsap, protocol s7comm, attack possible_mitm_observation;
threshold:type both, track by_src, count 60, seconds 10; priority:3;)
```

**From traffic to rule.** During Bettercap MITM (Section ??), the HMI still polls the PLC and receives Ack\_Data PDUs (02 F0 80 32 03) from port 102; the interceptor may rewrite payload bytes, but the header pattern stays valid. Counting more than 60 such responses in ten seconds from one PLC source flags unusually heavy read traffic toward the HMI subnet. The rule does not inspect modified values and cannot prove payload tampering—it supports correlation with sid:1000011 and L2/ARP alerts outside the S7comm rule set.

#### 0.4.5 Denial of service on the S7comm channel

Based on Section ??: script `s7comm_dos.py` (`-mode all`) floods Setup Communication, TCP connect churn, and Read SZL in sequence, with `--pause` between phases. Unlike single-opcode rules, the 100004x group in `ics_dos.rules` uses `threshold` to alert when one source exceeds a limit within ten seconds.

**Link to Chapter 3 traffic.** The DoS PCAP from Section ?? contains three distinguishable phases. Phase 1 (`setup_flood`) repeats the same Setup PDU as reconnaissance (0xF0 at offset 17, often on new TCP sessions) at high rate—Wireshark shows dense 0xF0 Jobs but each individual packet is valid. Phase 2 (`tcp_connect`) produces many TCP SYN packets to port 102 without completing S7comm. Phase 3 (`szl_flood`) reuses the Read SZL byte string from sid:1000002 (|00 01 12 04 11 44 01 00|) at 25+ times per ten seconds. A generic Job flood mixes multiple opcodes under 32 01. Thresholds were chosen from the script defaults (4–6 workers, 10 s phases) so legitimate HMI traffic stays below the counts in Table ?? of Chapter 3.

**Rule sid:1000040.**

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS possible TCP SYN flood to ISO-TSAP port 102";
flow:to_server; flags:S;
threshold:type both, track by_src, count 40, seconds 10; sid:1000040; rev:1;
metadata:attack dos_connection_flood; priority:1;)
```

**From traffic to rule.** The `tcp_connect` phase opens and closes TCP sessions rapidly; Wireshark filters on `tcp.flags.syn==1 tcp.dstport==102` show bursts of SYN packets without a full S7comm handshake. This rule counts SYN segments to \$HOME\_NET:102 per source and fires above 40 in ten seconds. It does not inspect S7 payload at all. Fast port scans or misconfigured clients can produce similar SYN rates, so the alert needs correlation with phases 1 or 3 on the same source IP.

#### Rule sid:1000041.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM possible Setup Communication flood to PLC";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5; content:"|F0|"; offset:17; depth:1;
  threshold:type both, track by_src, count 25, seconds 10; sid:1000041; rev:1;
  metadata:attack dos_s7comm_setup_flood; priority:1;)
```

**From traffic to rule.** Each packet in `setup_flood` is byte-identical to a normal Setup Communication—the same `|F0|` at offset 17 matched by `sid:1000001`. Wireshark cannot separate flood from a single setup on one packet alone. The threshold requires 25 or more Setup PDUs from one source in ten seconds, which the lab script produces but a single HMI session does not. Reconnect loops on a faulty client could approach the threshold and need baseline tuning.

#### Rule sid:1000042.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM possible Read SZL flood - PLC service load";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 07|"; offset:4; depth:5;
  content:"|00 01 12 04 11 44 01 00|"; offset:17; depth:8;
  threshold:type both, track by_src, count 25, seconds 10; sid:1000042; rev:1;
  metadata:attack dos_s7comm_szl_flood; priority:1;)
```

**From traffic to rule.** The `szl_flood` phase sends the same Read SZL User-data header observed in reconnaissance; only the rate differs. At 25+ requests per ten seconds the PLC CPU spends measurable time building SZL lists. The content match is copied from `sid:1000002`; the threshold distinguishes attack from a single Nmap `s7-info` run. Aggressive monitoring tools that poll SZL periodically could false-positive if their interval is misconfigured.

#### Rule sid:1000043.

```
alert tcp any any -> $HOME_NET 102 (msg:"ICS S7COMM possible S7 Job request flood to PLC";
  flow:to_server,established; content:"|03 00|"; offset:0; depth:2;
  content:"|02 F0 80 32 01|"; offset:4; depth:5;
  threshold:type both, track by_src, count 80, seconds 10; sid:1000043; rev:1;
  metadata:attack dos_s7comm_job_flood; priority:1;)
```

**From traffic to rule.** When the script mixes Setup, Write, or other Job opcodes under `ROSCTR=0x01`, no single function byte is stable enough for a narrow signature. Wireshark still shows `02 F0 80 32 01` on every Job flood packet. This rule counts 80 such PDUs per ten seconds without checking offset 17, covering `setup_flood` and generic Job spam. The higher count avoids overlap with `sid:1000041` on Setup-only floods but may miss slow-rate abuse below 80 packets.

## 0.4.6 Malformed packets and abnormal PDUs

Rules in `s7comm_malformed.rules` add a detection layer based on abnormal payload structure on TCP/102, not on dangerous opcodes alone. The design pairs with script `s7comm_malformed.py` (Section ??): each `--mode` injects one specific fault; the matching rule uses `content`, `byte_test`, or `dsize` at the offsets in Table ??.

**Link to Chapter 3 traffic.** Table ?? in Section ?? lists the exact byte change for each script mode against a valid Setup Communication template (0300001e02f08032010000050000140000f000000000000000000000000000000000). In Wireshark, `tpkt_bad_version` shows 02 00 instead of 03 00 at offset 0; length mismatch modes leave bytes 2–3 at 01 00 or 00 08 while the TCP segment is longer or shorter than declared; `cotp_invalid` replaces 02 F0 80 with 01 00 00; header modes change byte 7 to 0x31 or byte 8 to 0xFF; function modes set offset 17 to 0xFF or send a 20-byte Write Var stub; truncation sends only 14 bytes so offset 17 never arrives. Each fault is visible in the hex dump before the PLC closes the session.

**Group 1 — TPKT / COTP (sid:1000050, 1000051, 1000058, 1000056).** 1000050: `content:|03 00|` at offset 0—catches non-standard TPKT version (script sends 0x02 00). 1000051: after confirming 03 00, `byte_test` compares TPKT length (big-endian, offset 2) against `dsize`—fires when length is larger than the TCP payload (script sets 0x0100). 1000058: same idea but length is smaller than the actual payload (0x0008 with a full Setup body). 1000056: valid TPKT but missing COTP 02 F0 80 at offset 4.

**From traffic to rule.** Wireshark marks Group 1 faults at the outer encapsulation: wrong TPKT version breaks the dissector tree immediately; inflated or deflated length fields disagree with the TCP segment size shown in the status bar; missing COTP prevents S7comm parsing altogether. These rules use negated content, `byte_test`, or absence of 02 F0 80 rather than function opcodes because malformed packets may never reach offset 17. They can also match non-S7 traffic on port 102 if HOME\_NET is wide.

**Group 2 — S7 header (sid:1000052, 1000053).** 1000052: COTP DT is present but Protocol ID byte at offset 7 is not 0x32. 1000053: matches |02 F0 80 32 FF| at offset 4 (ROSCTR 0xFF from script `s7_bad_rosctr`); additional invalid ROSCTR values can be added with separate rules if needed.

**From traffic to rule.** Once TPKT and COTP look valid, bytes 7–8 still identify S7comm. The `s7_bad_protocol_id` PCAP shows 0x31 instead of 0x32 at offset 7; `s7_bad_rosctr` sets byte 8 to 0xFF, which Wireshark flags as an unknown PDU type. Matching these fixed patterns catches header fuzzing before any function code is evaluated. Valid engineering traffic always uses 0x32 and a known ROSCTR, so false positives are rare on pure S7 sessions.

**Group 3 — Function / parameters (sid:1000054, 1000057).** 1000054: Job (32 01) with function 0xFF at offset 17—opcode outside the lab command set. 1000057: still a Job with 0x05 (Write Var) but `dsize`<25: PDU too short for a valid Write Var (script `write_var_malformed`).

**From traffic to rule.** Group 3 packets pass header checks but fail semantic validation on the PLC. Wireshark may decode 0xFF at offset 17 as an unknown function; the truncated Write Var PDU shows 0x05 with no item structure and total length below 25 bytes. `sid:1000054` uses a content match on |FF| at offset 17; `sid:1000057` uses `dsize` because the anomaly is overall length rather

than a single byte value. Neither rule fires on the valid Write Var from the Stuxnet scenario.

**Group 4 — Truncation (sid:1000055).** 1000055: header 02 F0 80 32 01 is present but `dsize<18`—packet too short to contain the function byte at offset 17 (Setup truncated to 14 bytes). This is a “missing byte” signature independent of the specific opcode.

**From traffic to rule.** The `setup_truncated` mode sends only the first 14 bytes of a Setup PDU; Wireshark shows an incomplete S7comm tree and no byte at offset 17. `dsize<18` detects any Job header without a reachable function field, including random truncation by a proxy. It does not say which opcode was intended—only that the PDU is structurally incomplete.

**General note.** Malformed rules do not replace DoS rules (100004x): flooding valid packets still requires `threshold`. Conversely, some malformed packets will not trigger 1000001 (Setup) because 0xF0 is missing or offset 17 is unreachable. In the lab evaluation (Chapter ??), success is defined as a 100005x alert after running `--mode all`, without requiring the PLC to change RUN/STOP state.

Table 1: Byte offsets in the TCP/102 payload (S7comm Data TPDU)

| <b>Offset</b> | <b>Value</b> | <b>Field</b>                      |
|---------------|--------------|-----------------------------------|
| 0-1           | 03 00        | TPKT                              |
| 4-6           | 02 F0 80     | COTP Data TPDU                    |
| 7             | 32           | S7 protocol id                    |
| 8             | 01/03/07     | ROSCTR: Job / Ack_Data / Userdata |
| 17            | —            | Function code (Job, Userdata)     |
| 19            | —            | Function code (Ack_Data)          |